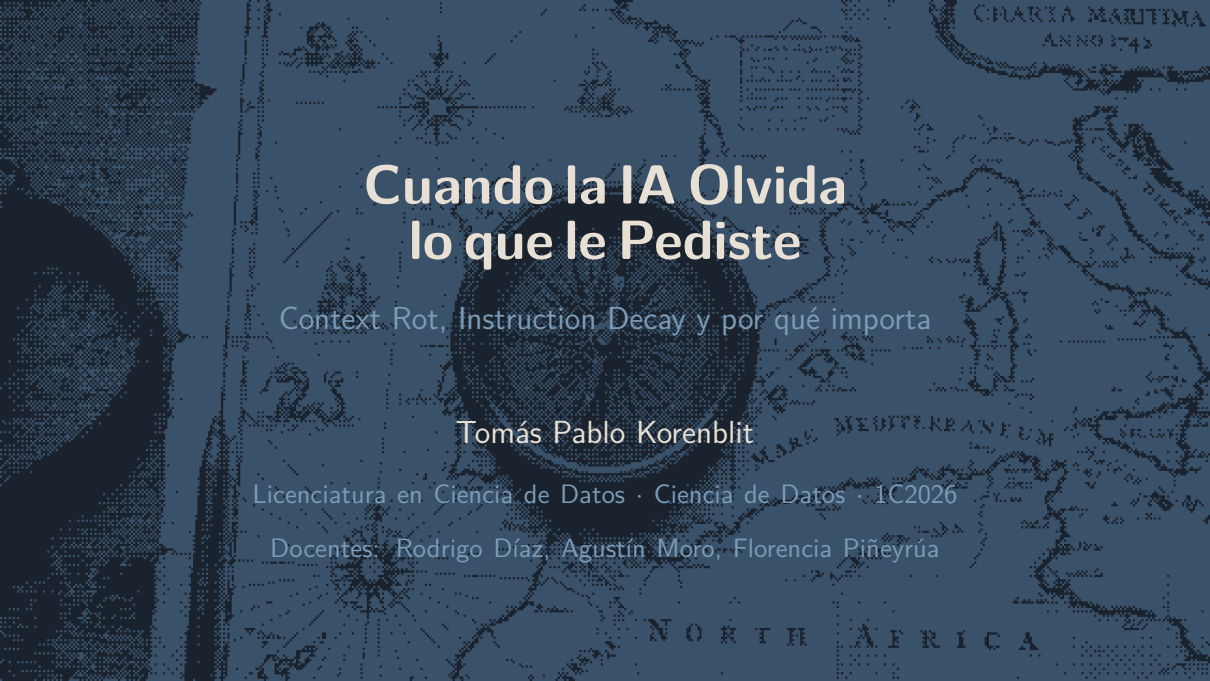




Cuando la IA Olvida lo que le Pediste

Context Rot, Instruction Decay y por qué importa

Tomás Pablo Korenblit

Licenciatura en Ciencia de Datos · Ciencia de Datos · 1C2026

Docentes: Rodrigo Díaz, Agustín Moro, Florencia Piñeyrúa

Estás usando ChatGPT.

Le explicás tu proyecto, le pedís que siga ciertas reglas, y funciona bárbaro. Van y vienen mensajes. Todo bien.

Estás usando ChatGPT.

Le explicás tu proyecto, le pedís que siga ciertas reglas, y funciona bárbaro. Van y vienen mensajes. Todo bien.

Hasta que en el mensaje 15...

te responde cualquier cosa, como si nunca le hubieras dicho nada.

Estás usando ChatGPT.

Le explicás tu proyecto, le pedís que siga ciertas reglas, y funciona bárbaro. Van y vienen mensajes. Todo bien.

Hasta que en el mensaje 15...

te responde cualquier cosa, como si nunca le hubieras dicho nada.

¿Qué pasó?

El Campo: ¿Cómo Funciona un LLM?

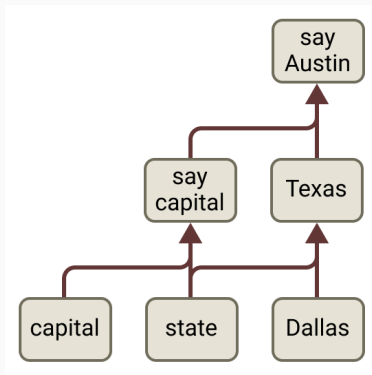
Un LLM es una Máquina de Predicción

Un Large Language Model no “entiende”. Predice: dado todo lo anterior, ¿qué palabra viene?

Pero esa predicción es sofisticada. El año pasado, Anthropic le hizo reverse engineering a uno de sus modelos y publicó lo que encontró adentro (Lindsey et al., 2025):

- No procesa paso a paso. Evalúa **múltiples caminos en paralelo** y “vota”
- Planifica hacia adelante: elige una palabra futura *antes* de escribir lo que viene antes

¿Cómo lo hace? Con **circuitos internos**.



“¿Cuál es la capital del estado donde está Dallas?”

El modelo encadena representaciones internas:

Dallas → **Texas** → **capital** → **Austin**

Cada nodo es una *representación interna* (no confundir con feature de un dataset) que se activa según el contexto.

Lindsey et al., 2025. Transformer Circuits.

La atención: lo que el modelo “mira”

Para generar cada palabra, el modelo decide **a qué parte del texto prestarle atención**.

Funciona bien cuando el contexto es corto: hay pocos tokens y el modelo puede atender a cada uno.

Pero la atención es un **recurso finito**. Se reparte entre *todos* los tokens del contexto. Cuando el contexto crece, las instrucciones del principio tienen que competir con todo lo que vino después.

¿Qué pasa cuando hay **un millón de tokens** compitiendo por atención?

Token = la unidad mínima que procesa el modelo.

Una palabra $\approx$ 1–2 tokens. Una página $\approx$ 500.

Ventana de contexto = todo lo que el modelo puede “ver” al responder: el system prompt, la conversación, los archivos.

Si algo **no está en la ventana**, el modelo **no lo sabe**. No tiene memoria aparte.

Año	Contexto
2023	4K–16K
2024	128K–200K
2025	200K
2026	1M tokens

1M tokens $\approx$ 2.000 páginas.

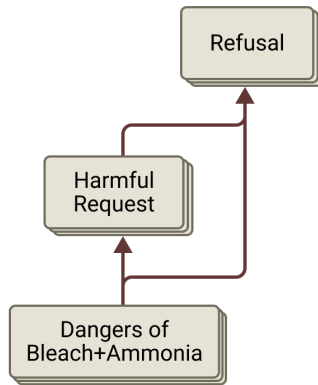
Más ventana debería ser mejor, ¿no?

System prompt y guardrails

System prompt = instrucciones que recibe el modelo *antes* de hablar con el usuario.

- **Formato** — “respondé en español”
- **Persona** — “sos un tutor amable”
- **Seguridad** — “no generes contenido dañino”
- **Lógica** — “usá tal herramienta para tal tarea”

Los **guardrails** no son un módulo especial. Son instrucciones como cualquier otra.



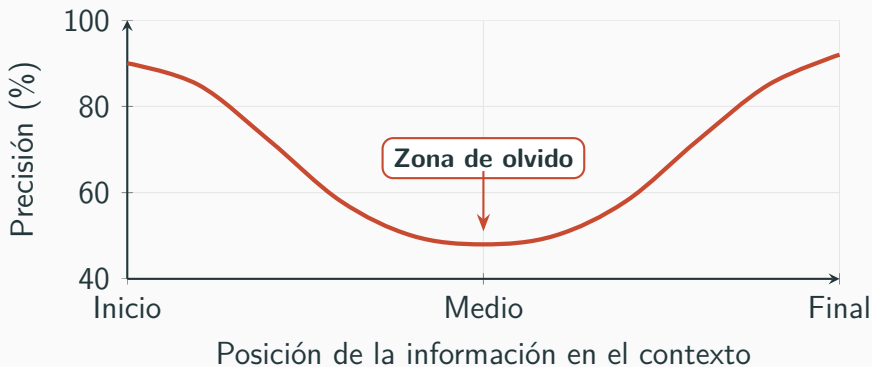
La seguridad es un *pathway*, no un interruptor. (Lindsey et al., 2025)

El Problema: Context Rot

Hallazgo	Número	Fuente
Caída en tareas multi-turn	39%	Laban et al., 2025
Accuracy en 3 turnos: 87.7% →	70.7%	He et al., 2024
Modelos frontera donde se observa	18/18	Chroma, 2025
Con 50+ reglas, adherencia →	~0%	Mu et al., 2025

No es un bug de un modelo. Le pasa a **todos**: los 18 modelos frontera testeados.

¿Por qué pasa? La Curva U



Liu et al. (2023) midió esto en tareas de recuperación de información. El patrón sugiere que la atención no es uniforme, lo que podría explicar por qué las instrucciones también se pierden.

¿Y por qué debería importarnos?

Plata:

- Empresas procesan **miles de millones** de tokens por día
- Cada token de reminder cuesta
- Refuerzo ineficiente $\times$ esa escala
= mucha plata

Seguridad:

- Los guardrails son instrucciones como cualquier otra
- **Hipótesis:** seguridad podría decaer más rápido que formato
- Si es así, los reminders uniformes **sub-invierten en seguridad**

Le pedís que solo cite fuentes reales. 10 turnos después, te inventa un paper que no existe.

¿Qué se Hace Hoy?

Estrategia	Idea	Problema
Repetir todo, siempre	Re-inyectar el prompt completo cada turno	Saturación: más reglas → <i>menos</i> adherencia
Duplicar el prompt	Poner las instrucciones dos veces (Google, 2025)	2× costo en tokens
No hacer nada	Confiar en que el modelo recuerda	39% de caída documentada
Jerarquía de prioridad	System > User (OpenAI, 2024)	No aborda el decaimiento gradual

Ninguna es **adaptativa**. Ninguna decide *qué* reforzar ni *cuándo*.

Si sabemos que el modelo **olvida...**
y sospechamos que cada instrucción **decae distinto...**

Si sabemos que el modelo **olvida...**
y sospechamos que cada instrucción **decae distinto...**
¿por qué reforzamos **todo por igual?**

Si sabemos que el modelo **olvida...**
y sospechamos que cada instrucción **decae distinto...**

¿por qué reforzamos **todo por igual?**

¿Y si pudiéramos reforzar **solo lo que se está por olvidar?**

Una Dirección: Pensar Bayesianamente

Enfoque clásico:

“¿Cumple o no cumple?” — binario.

Si cumple, no hacemos nada.

Si no cumple, ya es tarde.

No distingue entre una instrucción que está al 90% y una al 51%.

Enfoque Bayesiano:

Mantenemos un *posterior* para cada instrucción.

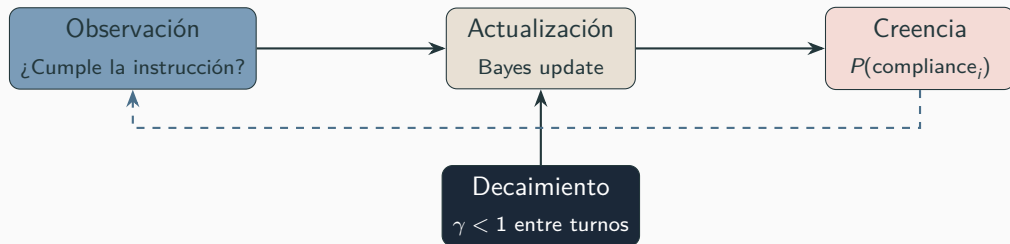
Cada observación **actualiza** la estimación (verosimilitud $\times$ prior).

Entre turnos, el posterior **decae** naturalmente.

Podemos **anticipar** el olvido antes de que pase.

Bayesian Knowledge Tracing (BKT)

BKT se usa hace 30 años en educación: estima la probabilidad de que un **estudiante** domine un concepto, turno a turno.



¿Qué pasa si el “estudiante” es el **LLM** y los “conceptos” son sus **instrucciones**?

Para cada instrucción i en el turno t :

$$P(C_i | \mathcal{O}_{1:t}) = \frac{P(\mathcal{O}_t | C_i) \cdot \overbrace{P(C_i | \mathcal{O}_{1:t-1})}^{\text{lo que creíamos antes}}}{P(\mathcal{O}_t)}$$

- C_i = la instrucción i sigue siendo cumplida
- $\gamma < 1$ controla cuánto decae entre turnos
- Con presupuesto fijo de B tokens, se refuerzan solo las instrucciones con menor $P(C_i)$

Nadie aplicó BKT a compliance de LLMs, ni midió curvas de olvido por tipo de instrucción, ni optimizó qué reforzar con un presupuesto limitado de tokens.

¿Preguntas?

Referencias

Lindsey et al. (2025). *On the Biology of a Large Language Model*. Transformer Circuits.

Ameisen et al. (2025). *Circuit Tracing*. Transformer Circuits.

Anthropic (2025). *Activation Oracles*.
alignment.anthropic.com.

Liu et al. (2024). *Lost in the Middle*. TACL.

Chroma Research (2025). *Context Rot*.

Du et al. (2025). *Context Length Alone Hurts*. EMNLP.

Laban et al. (2025). *LLMs Get Lost in Multi-Turn Conversation*.

He et al. (2024). *Multi-IF*. Meta.

Mu et al. (2025). *System Prompt Robustness*.

Zhang & Yang (2025). *LLMs as Discounted Bayesian Filters*.

Google Research (2025). *Prompt Repetition Improves Non-Reasoning LLMs*.

Wallace et al. (2024). *The Instruction Hierarchy*. OpenAI.